KEVIN C. JONES - 4N6POST.COM

Linux Commands - Manipulation Cheat Sheet

Core Unix/Linux text tools for filtering, slicing, transforming, sorting, and reporting: `grep`, `awk`, `cut`, `tr`, `rev`, `pr`, `sed`, and friends - with simple, medium, and complex examples for log and data analysis.

Tool Map - Pick the Right Utility 1

NEED	TOOL	ROLE
Find lines by pattern	`grep` (`-E` / `-F`)	Regex or fixed-string search; context lines; invert match. Legacy aliases: `egrep`/`fgrep`.
Field logic / math / report	`awk`	Pattern-action language: field processing, filtering, arithmetic, aggregation, and simple joins.
Extract columns by delimiter	`cut`	Fast field/byte/char slices; no computation; not a CSV parser.
Translate / delete chars	`tr`	Character sets: case, squeeze, delete, complement (stdin only).
Reverse characters	`rev`	Reverse each line; last-field tricks need care (prefer `awk`/`basename`).
Paginate / multi-column print	`pr`	Headers, columns, page layout for printing/reports.
Stream edit / substitute	`sed`	In-place or pipe replace, delete, insert lines.
Order / unique / count	`sort` + `uniq`	Sort keys; collapse duplicates; frequency counts.
Window of a file	`head` / `tail`	First/last N lines; follow growing logs (`-f`).
Count lines/words/bytes	`wc`	Quick volume metrics.
Merge columns / files	`paste` / `join`	Side-by-side paste; key-based relational join.
Pretty columns / wrap	`column` / `fold` / `fmt`	Align tables; hard-wrap; reflow paragraphs.
Number / reverse lines	`nl` / `tac`	Line numbers; reverse line order (not chars).
Batch args / tee output	`xargs` / `tee`	Build command lines from stdin; split stream to file+stdout.
Compare sorted lists	`comm` / `diff`	Set ops on sorted files; unified diffs.

Windowing Examples 17

LEVEL	COMMAND / RESULT
SIMPLE	`head -n 5 file`
SIMPLE	`wc -l *.log`
MEDIUM	`tail -n +2 data.csv | _` - skip header.
MEDIUM	`tail -F /var/log/syslog | grep --line-buffered error` - GNU `tail -F + --line-buffered`.
COMPLEX	`sed -n '100,200p' big.log | tee slice.log | wc -l`

grep - Options 2

FLAG	MEANING
`-i`	Ignore case.
`-v`	Invert match (non-matching lines).
`-n`	Show line numbers.
`-H` / `-h`	With / without filename prefix.
`-c`	Count matching lines only.
`-l` / `-L`	List files with / without matches.
`-o`	Print only the matched part.
`-w`	Whole-word match.
`-x`	Whole-line match.
`-E`	Extended regex (prefer over legacy `egrep`).
`-F`	Fixed strings (prefer over legacy `fgrep`; often faster).
`-P`	PCRE-style regex on GNU grep when built with PCRE; not portable.
`-A`/`-B`/`-C N`	After / before / context lines.
`-r` / `-R`	Recursive (symlink handling differs by implementation).
`--include=`	Glob filter when recursing (GNU).
`-m N`	Stop after N matches per file.
`-a`	Treat binary as text.

Exit: `0` match, `1` no match, `2` error.

grep - Examples 3

LEVEL	COMMAND	WHAT IT DOES
SIMPLE	`grep error app.log`	Lines containing `error`.
SIMPLE	`grep -in fail *.log`	Case-insensitive + line numbers across logs.
SIMPLE	`grep -vE '^[[:space:]]*#|^[[:space:]]*$' config.ini`	Drop blank lines and full-line comments (optional leading space).
MEDIUM	`grep -E 'failed|denied|invalid' auth.log`	Any of several failure keywords.
MEDIUM	`grep -nC 3 'segfault' /var/log/syslog`	Match plus 3 lines of context.
MEDIUM	`grep -r --include='*.log' -n '10\.0\.0\.' /var/log`	Recursive IP-ish pattern in `*.log` only (GNU `--include`).
MEDIUM	`grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' access.log | sort -u`	Unique IPv4-looking tokens (does not validate 0-255 octets).
COMPLEX	`grep -E 'sshd.*(Failed|Invalid).*from' auth.log | grep -oE 'from ([0-9]{1,3}\.){3}[0-9]{1,3}' | awk '{print $2}' | sort | uniq -c | sort -nr | head`	Top approximate source IPs for SSH failures (IPv4-looking only).
COMPLEX	`grep -P '^(?=.*ERROR)(?!.*retry).*$' app.log`	PCRE lookaround: ERROR lines that are not retries (GNU + PCRE build only).

awk - Model & Built-ins 4

ITEM	MEANING
Form	`pattern { action }`
`$0`	Whole line.
`$1..$n`	Fields (1-based).
`NF`	Number of fields.
`NR` / `FNR`	Record # overall / per file.
`FS` / `-F`	Input field separator.
`OFS`	Output field separator.
`RS` / `ORS`	Record separators.
`BEGIN` / `END`	Run before / after input.
`length()`	String or field length.
`split(s,a,fs)`	Split string into array.
`substr(s,i,n)`	Substring.
`gsub`/`sub`	Global / first substitute.
`printf`	Formatted print.
`next`	Skip to next record.

`awk -F: -v OFS='\t' '…' file`

awk - Examples 5

LEVEL	COMMAND	WHAT IT DOES
SIMPLE	`awk '{print $1}' file`	Print first whitespace field.
SIMPLE	`awk -F, '{print $2,$4}' data.csv`	Simple comma-delimited cols 2 and 4 (not quoted CSV).
SIMPLE	`awk 'NR==1 || $3>100' tsv`	Header + rows where col3 > 100.
MEDIUM	`awk -F: '{print $1":"$3":"$7}' /etc/passwd`	User, UID, shell.
MEDIUM	`awk 'NR {sum+=$NF} END {if (NR) print sum, NR, sum/NR; else print 0,0,"n/a"}' sizes.txt`	Sum last field; count; average (guards empty input).
MEDIUM	`awk -F'\t' 'NF!=12 {print FILENAME":"FNR": bad NF="NF}' *.tsv`	Flag malformed TSV rows.
MEDIUM	`awk '/^ERROR/ {e++} /^WARN/ {w++} END {print e,w}' app.log`	Count ERROR vs WARN lines.
COMPLEX	`awk -F'\t' 'NR>1 {bytes[$3]+=$5; n[$3]++} END {for (ip in bytes) printf "%s\t%d\t%.0f\n", ip, n[ip], bytes[ip]}' conn.tsv \| sort -t "$(printf '\t')" -k3,3nr | head`	Per-IP count + total bytes; top talkers (tab-separated output; Bash `$'\t'` also fine).
COMPLEX	`awk 'BEGIN{FS=OFS=","} {gsub(/^[ \t]+|[ \t]+$/,"",$2); if($2~/^[0-9]+(\.[0-9]+)?$/) print $1,$2*1.1}' prices.csv`	Trim field, validate a plain numeric value (e.g., `123.45`), raise price 10% (simple CSV).

head · tail · wc 16

COMMAND	MEANING
`head -n 20`	First 20 lines.
`head -c 1024`	First 1024 bytes (`1K` suffix is GNU).
`tail -n 50`	Last 50 lines.
`tail -n +100`	From line 100 to EOF.
`tail -f`	Follow appends (live log).
`tail -F`	Follow + retry on rotate (GNU).
`wc -l`	Line count.
`wc -w`	Word count.
`wc -c` / `-m`	Bytes / characters.
`wc -L`	Longest line length (GNU).

cut - Options 6

FLAG	MEANING
`-d DELIM`	Field delimiter (default TAB).
`-f LIST`	Fields: `1,3` or `2-5` or `3-`.
`-c LIST`	Character positions (locale-dependent; use `LC_ALL=C` for bytes-as-chars).
`-b LIST`	Byte positions (safer for binary/ASCII offsets).
`--complement`	Invert the selection (GNU).
`-s`	Skip lines without delimiter.
`--output-delimiter=`	Join selected fields with string.

No regex; not a CSV parser. Use `awk` for multi-space collapse or logic.

cut - Examples 7

LEVEL	COMMAND / RESULT
SIMPLE	`cut -d: -f1 /etc/passwd` - usernames.
SIMPLE	`LC_ALL=C cut -c1-10 file` - first 10 chars (C locale = bytes on ASCII).
MEDIUM	`cut -d, -f1,4-6 --output-delimiter=$'\t' data.csv` - simple CSV; GNU/Bash $'\t'.
MEDIUM	`cut -d' ' -f1-3 access.log` - may break on multi-space; prefer awk.
COMPLEX	`basename /path/to/file` or `awk -F/ '{print $NF}'` - last path segment (prefer over `rev|cut|rev`, which breaks on trailing `/`).
COMPLEX	`grep -oE 'uid=[0-9]+' auth.log | cut -d= -f2 | sort -n | uniq -c`

Regex Quick Reference 24

TOKEN	MEANING
`.`	Any one character.
`^` / `$`	Start / end of line.
`*` `+` `?`	0+, 1+ (ERE), 0 - 1 (ERE).
`{n,m}`	Count range (ERE).
`[abc]` `[^a]`	Class / negated.
`[0-9]` / `[[:digit:]]`	Digit. PCRE-only: `\d`.
`|`	Alternation (ERE).
`(…)`	Group / capture (ERE).
`\b`	Word boundary (engine-dependent; not portable BRE).
`[[:space:]]` / `[[:alnum:]_]`	Space / word-ish classes (portable). PCRE: `\s` `\w`.

BRE (basic): escape `\( \) \{ \}`. Prefer `grep -E` / `sed -E`. Avoid `\s`/`\d` unless using `grep -P`.

rev & pr 10

TOOL	USE
`rev`	Reverse character order on each line (not always installed; not for path parsing).
`pr`	Format text for printing; pages, columns, headers (not a general pretty-printer).
`pr -n`	Number lines (for print layout; daily use: prefer `nl`).
`pr -2` / `-3`	Multi-column layout.
`pr -t`	Omit page headers/trailers.
`pr -w N`	Page width.
`pr -h "title"`	Custom header.
`pr -m a b`	Merge files side by side.

rev / pr - Examples 11

LEVEL	COMMAND / RESULT
SIMPLE	`echo hello | rev` → `olleh`
SIMPLE	`pr -n file | less` - numbered pages.
MEDIUM	`pr -t -e8 file` - expand tabs to 8 spaces, no header.
MEDIUM	`pr -m -t left.txt right.txt` - side-by-side.
COMPLEX	`awk '{n=split($1,a,"."); if(n) print a[n]}' access.log` - last DNS label of field1 (clearer than `rev|cut|rev`).
COMPLEX	`pr -3 -t -w 120 list.txt | expand` - 3-col directory-style dump.

tr - Options & Sets 8

FLAG / SET	MEANING
`tr SET1 SET2`	Map chars in SET1 → SET2; if SET1 and SET2 differ in length, the last char of SET2 is often repeated by GNU tr. This is implementation-dependent, so avoid relying on it for exact-length mapping.
`-d SET`	Delete characters in SET.
`-s SET`	Squeeze repeats to one.
`-c` / `-C`	Complement of SET1.
`[:lower:]`	Lowercase letters.
`[:upper:]`	Uppercase letters.
`[:digit:]`	Digits 0 - 9.
`[:space:]`	Whitespace.
`[:punct:]`	Punctuation.
`[:alnum:]`	Letters + digits.
`\n` `\t`	Newline, tab (quote carefully).

`tr` reads stdin only - no filename args. Ranges like `a-z` are locale-sensitive; prefer `[:lower:]`/`[:upper:]` or `LC_ALL=C`.

tr - Examples 9

LEVEL	COMMAND	WHAT IT DOES
SIMPLE	`tr '[:lower:]' '[:upper:]' < file`	Uppercase letters (portable classes).
SIMPLE	`tr -d '\r' < dos.txt > unix.txt`	Strip CR (DOS → Unix).
SIMPLE	`tr -s ' ' < file`	Squeeze repeated spaces.
MEDIUM	`tr ',;' '\t\t' < messy.csv`	Map `,` and `;` to tabs (not a CSV parser; ignores quotes).
MEDIUM	`tr -cd '[:alnum:]\n' < file`	Keep only alnum + newlines.
MEDIUM	`tr -s '[:space:]' '\n' < words | sort -u`	Whitespace → newline chars (GNU tr interprets `\n` in SET2 as newline; shell quoting can vary by platform).
COMPLEX	`tr -cs '[:alnum:]' '\n' < corpus.txt | tr '[:upper:]' '[:lower:]' | sort | uniq -c | sort -nr | head -20`	Word-frequency top 20.
COMPLEX	`xxd -p file.bin | tr -d '\n' | fold -w2 | …`	Hex stream prep (pair with fold/xxd).

sed - Essentials 12

FORM	MEANING
`s/old/new/`	Substitute first match.
`s/old/new/g`	Global on line.
`s/old/new/i`	Case-insensitive substitute (GNU; not POSIX).
`s//&/`	`&` = whole match.
`\1..\9`	Capture groups.
`d`	Delete line.
`p`	Print (with `-n`).
`-n`	Quiet; only explicit print.
`-i.bak`	In-place edit + backup (GNU; BSD syntax differs).
`-E` / `-r`	Extended regex (`-i` preferred; `-r` GNU synonym).
`addr1,addr2 cmd`	Address range.
`/pat/cmd`	On matching lines only.

sed - Examples 13

LEVEL	COMMAND	WHAT IT DOES
SIMPLE	`sed 's/foo/bar/' file`	First `foo` → `bar` per line.
SIMPLE	`sed '/^#/d;/^$/d' file`	Drop comments and blank lines.
MEDIUM	`sed -n '10,25p' file`	Print lines 10 - 25 only.
MEDIUM	`sed -E 's/([0-9]{1,3}\.){3}[0-9]{1,3}/<IP>/g' log`	Redact IPv4-looking strings (not range-validated).
MEDIUM	`sed -i.bak 's/\t/|/g' data.tsv`	Tabs → pipes; keep `.bak` (GNU `-i`).
COMPLEX	`sed -nE 's/.*src=([^ ]+).*dst=([^ ]+).*/\1 -> \2/p' fw.log`	Extract src/dst pairs from key=value log.
COMPLEX	`sed -E '/BEGIN/{:a;N;/END/!ba;s/\n/ /g}' multi.txt`	Join multi-line blocks between BEGIN/END (GNU sed labels/branching).

paste · join · split 18

TOOL	ROLE
`paste a b`	Merge lines side-by-side (TAB).
`paste -d,`	Custom join delimiter.
`paste -s`	Serialize file to one line.
`join`	Relational join on sorted key field.
`join -t,`	Comma delimiter (simple CSV only; inputs must be sorted on key).
`join -1 2 -2 1`	Key fields per file.
`join -a1`	Left outer join (GNU/Bash-specific flags such as `-e` and `-o auto` are not POSIX).
`split -l 1000`	Chunk by lines.
`split -b 50M`	Chunk by size.
`csplit`	Split on patterns/lines (feature set often GNU-leaning).

paste / join / split - Examples 19

LEVEL	COMMAND	WHAT IT DOES
SIMPLE	`paste names.txt ids.txt`	Two columns from two files.
SIMPLE	`split -l 500 big.csv part_`	500-line chunks: `part_aa`,…
MEDIUM	`paste -d, <(cut -f1 a) <(cut -f3 b)`	Build simple CSV from process substitutions (Bash).
MEDIUM	`join -t, <(sort -t, -k1,1 users.csv) <(sort -t, -k1,1 orders.csv)`	Inner join on col1 (Bash; simple unquoted CSV).
COMPLEX	`join -t$'\t' -a1 -e MISSING -o auto <(sort -k1,1 left.tsv) <(sort -k1,1 right.tsv)`	Left join; fill missing with `MISSING` (GNU/Bash-specific; `-e` and `-o auto` are GNU extensions).
COMPLEX	`csplit -f chunk -b '%02d.txt' report.txt '/^====/' '{*}'`	Split report on separator lines (GNU `csplit` `{*}` / `-b`).

sort & uniq 14

FLAG	MEANING
`sort -n`	Numeric sort.
`sort -r`	Reverse.
`sort -u`	Unique while sorting.
`sort -k2,2`	Key = field 2 only (fields split on blank runs by default).
`sort -t,`	Field separator (single char; simple CSV only).
`sort -h`	Human sizes (1K, 2M) - GNU.
`sort -V`	Version sort - GNU.
`sort -s`	Stable sort.
`uniq -c`	Count adjacent dups.
`uniq -d`	Only duplicated lines.
`uniq -u`	Only unique lines.
`uniq -i`	Ignore case.

`uniq` only collapses adjacent duplicates - sort first for global uniqueness. Set intersection: prefer `comm -12` on sorted unique lists.

sort / uniq - Examples 15

LEVEL	COMMAND	WHAT IT DOES
SIMPLE	`sort file | uniq`	Sorted unique lines.
SIMPLE	`sort -nr nums.txt`	Numeric descending.
MEDIUM	`cut -d: -f7 /etc/passwd | sort | uniq -c | sort -nr`	Shell popularity ranking.
MEDIUM	`sort -t, -k3,3n -k1,1 data.csv`	Sort by col3 numeric, then col1 (simple unquoted CSV).
COMPLEX	`awk '{print $1}' access.log | sort | uniq -c | sort -nr | awk '$1>100'`	IPs (or field1) seen more than 100 times.
COMPLEX	`comm -12 <(sort -u a.txt) <(sort -u b.txt)`	True intersection of unique lines (Bash process substitution; both inputs sorted).

column · fold · fmt · nl · tac 20

COMMAND	MEANING
`column -t`	Align whitespace table.
`column -s, -t`	Comma → aligned table (not a quoted-CSV parser; util-linux/BSD variants differ).
`fold -w 80`	Hard-wrap at width.
`fold -s`	Break at spaces.
`fmt -w 72`	Reflow paragraphs.
`nl -ba`	Number all lines.
`nl -bt`	Number non-empty only.
`tac`	Reverse line order (GNU; BSD may lack it).
`expand` / `unexpand`	Tabs ↔ spaces.
`seq 1 10`	Generate number list.

Format Examples 21

LEVEL	COMMAND / RESULT
SIMPLE	`column -t -s: /etc/passwd | less -S`
SIMPLE	`nl -ba script.sh`
MEDIUM	`tac log | head -20` - newest-first if appended log.
MEDIUM	`fmt -w 60 notes.md`
COMPLEX	`awk -F, '{print $1,$3,$5}' f.csv | column -t | pr -h 'Report' -F` - simple CSV fields → table → print layout.

xargs · tee · comm · diff 22

FLAG / FORM	MEANING
`xargs cmd`	Append stdin tokens as args.
`xargs -n1`	One arg per invocation.
`xargs -I{}`	Replace `{}` placeholder.
`xargs -0` / `-d`	NUL delimiter (safe paths); `-d` custom delim (GNU).
`xargs -P N`	Parallel jobs (GNU). Prefer `-print0`/`-0` for filenames; use `-r` to skip runs when no input remains.
`tee file`	Stdout + write file.
`tee -a`	Append.
`comm -12 a b`	Lines common to both (sorted).
`comm -23 a b`	Only in a.
`diff -u`	Unified diff.
`diff -y`	Side-by-side.

xargs / tee / comm - Examples 23

LEVEL	COMMAND	WHAT IT DOES
SIMPLE	`find . -maxdepth 1 -name '*.log' -print0 | xargs -0 wc -l`	Line counts; NUL-safe (avoids `ls | xargs` pitfalls).
SIMPLE	`cmd | tee out.txt`	See output and save it.
MEDIUM	`find . -name '*.tmp' -print0 | xargs -0 rm -f`	NUL-safe delete - destructive; dry-run with `echo` first. Prefer `find … -delete` when available.
MEDIUM	`comm -12 <(sort -u a) <(sort -u b)`	Intersection of unique lines (Bash process substitution).
COMPLEX	`grep -rl 'TODO' src | tee todo-files.txt | xargs -I{} sh -c 'echo === {}; grep -n TODO "{}"'`	List files, save list, print each TODO with line #.
COMPLEX	`diff -u <(sort old.iocs) <(sort new.iocs) | tee ioc.diff | grep '^+[^+]'`	Added lines only (skip `+++` header); Bash process substitution.

Pipeline Recipes (Simple → Complex) 26

LEVEL	RECIPE	GOAL
SIMPLE	`grep -i error app.log | wc -l`	How many error lines?
SIMPLE	`cut -d, -f2 data.csv | sort | uniq -c | sort -nr | head`	Top values in column 2 (simple unquoted CSV).
SIMPLE	`tr '[:upper:]' '[:lower:]' < names.txt | sort -u`	Case-fold unique names.
MEDIUM	`awk -F'\t' 'NR>1 {print $4}' zeek/http.log | sort | uniq -c | sort -nr | head -15 | column -t`	Top HTTP hosts/fields from TSV.
MEDIUM	`grep -R --include='*.conf' -n 'password' /etc 2>/dev/null | tee hits.txt | cut -d: -f1 | sort -u`	Conf files mentioning password (lab/authorized hosts only; broad `/etc` search can be noisy/sensitive).
MEDIUM	`sed '1d' data.csv | cut -d, -f1,3 | tr ',' '\t' | sort -k2,2nr | pr -h 'Col1 by Col3' -F | head -40`	Skip header, rank, print layout (simple CSV; `pr` is for page layout).
COMPLEX	`zcat access.log.gz | awk '$9 ~ /^5/ {print $1}' | sort | uniq -c | sort -nr | awk 'BEGIN{print "count\tip"} {print $1"\t"$2}' | tee top5xx.tsv | column -t | head`	Top clients causing HTTP 5xx from gzipped combined-style log (`$9`=status; field layout varies).
COMPLEX	`grep -hE 'Failed password|Invalid user' /var/log/auth.log* | grep -oE 'from ([0-9]{1,3}\.){3}[0-9]{1,3}' | awk '{print $2}' | sort | uniq -c | sort -nr | awk '$1>=10 {print}' | tee brute_ips.txt | wc -l`	SSH brute sources with ≥10 hits; count IPs (IPv4-looking approximate).
COMPLEX	`join -t "$(printf '\t')" <(awk -F'\t' 'NR>1{print $2"\t"$1}' conn.log | sort) <(awk -F'\t' 'NR>1{print $2"\t"$10}' dns.log | sort) | awk -F'\t' '{c[$2]++} END{for (q in c) print c[q], q}' | sort -nr | head`	Join Zeek-style TSVs on uid; rank names (illustrative; Bash `<(…)`; field indexes vary by Zeek TSV).
COMPLEX	`find . -type f -name '*.log' -print0 | xargs -0 grep -l 'CRITICAL' | tee critical-files.txt | xargs -I{} sh -c 'echo; echo "{}"; grep -nC2 CRITICAL "{}"' | pr -h 'CRITICAL hits' -F | less`	All CRITICAL contexts across tree; paged report. NUL-safe, but still watch filenames with spaces/newlines and leading dashes before invoking `sh -c`.

Compose left → right: filter early (`grep`/`awk`), reshape (`cut`/`tr`/`sed`), aggregate (`sort`\|`uniq`/`awk`), present (`column`/`pr`/`tee`).

One-Liner Index (Copy/Paste) 27

TASK	ONE-LINER
Strip blank + comments	`sed '/^[[:space:]]*#/d;/^[[:space:]]*$/d' file`
Fields 1 and last	`awk '{print $1, $NF}' file`
Replace spaces → underscores	`tr ' ' '_' < file` (single spaces only; not tabs)
DOS → Unix newlines	`tr -d '\r' < dos > unix`
Unique sorted IPs from text	`grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' f | sort -u` (IPv4-looking)
Frequency table	`sort f | uniq -c | sort -nr`
Skip CSV header	`tail -n +2 file.csv`
Last path component	`basename "$path"` (portable; GNU/BSD also accept `--`) or `awk -F/ '{print $NF}'`
In-place safe sed	`sed -i.bak 's/old/new/g' file` (GNU; BSD differs)
Multi-file line total	`wc -l -- *.log` (avoid useless `cat | wc`)
Live filter log	`tail -F app.log | grep --line-buffered -i error` (GNU)
Aligned CSV preview	`head -20 f.csv | column -s, -t | less -S` (simple CSV)
Intersection of lists	`comm -12 <(sort -u a) <(sort -u b)` (Bash)
Only in a not b	`comm -23 <(sort -u a) <(sort -u b)` (Bash)
Parallel grep files	`find . -name '*.txt' -print0 | xargs -0 -r -P4 grep -H -- pattern` (GNU `-P` + GNU `-r`)
Number non-empty	`nl -bt file`
Squeeze whitespace	`tr -s '[:space:]' < f`
Print every 10th line	`awk 'NR%10==0' file`
Sum a column	`awk -F, '{s+=$3} END{print s+0}' f.csv` (simple CSV)
Dual output	`… | tee keep.txt | grep keep_only`